IaC Provisioning for Finance System

Course Name: DevOps Foundation

Institution Name: Medicaps University-Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	ADITYA DUBEY	EN22CS301050
2.	ADITYA PARMAR	EN22CS301058
3.	ADITYA VYAS	EN22CS301074
4.	ALOK DALKE	EN22CS301103
5.	ADITYA PRATAP SINGH PARIHAR	EN22CS301061

Group Name: Group 03D1

Project Number: DO-03

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Problem Statement & Objectives

1. **Problem Statement:** IaC Provisioning for Finance System

2. **Project Objectives:** Automate the provisioning of infrastructure for a Finance system using Infrastructure as Code (IaC) tools like Terraform or Ansible. The project involves script-based deployment of local Docker environments or cloud instances (e.g., AWS), including the automated installation of core dependencies such as Terraform or Ansible. This approach eliminates manual configuration errors and enables repeatable, scalable environment setup.

3. **Scope of the Project:** The scope of this project is to design and implement an automated deployment system for a Finance application using Infrastructure as Code (IaC) and modern DevOps practices. The project involves developing a Finance web application using the Flask framework and containerizing it with Docker to ensure a consistent and portable runtime environment across local and cloud platforms.

   The infrastructure required for hosting the application is provisioned automatically on Amazon Web Services (AWS) using Terraform. Terraform scripts define and create cloud resources such as Virtual Private Cloud (VPC), public subnet, internet gateway, security group, and EC2 instance, enabling repeatable and error-free infrastructure setup through code rather than manual configuration.

   The project also includes automated application deployment using EC2 user-data scripts, which install Docker, pull the application image from Docker Hub, and run the container during instance initialization. Continuous Integration and Continuous Deployment (CI/CD) is implemented using GitHub Actions, allowing automatic building, testing, and deployment of the application whenever code changes are pushed to the repository.

Additionally, the project scope extends to future scalability and architectural improvements as part of High-Level Design (HLD) expansion, including load balancing, auto scaling, improved security practices, and highly available cloud deployment. The overall system demonstrates an efficient, scalable, and reliable automated deployment workflow suitable for modern cloud-based applications.

Proposed Solution

1. Key Features:

- Automated infrastructure provisioning using Infrastructure as Code (IaC)
- Finance web application deployment using Docker containers
- AWS cloud infrastructure setup using Terraform
- Automatic application deployment through EC2 user-data scripts
- CI/CD pipeline for automated build and deployment
- Reusable and scalable infrastructure modules
- Reduced manual configuration and deployment errors

2. Overall Architecture:

The proposed solution follows an automated DevOps architecture for deploying a Finance application using containerization, Infrastructure as Code (IaC), and CI/CD automation. The system ensures fast, consistent, and error-free deployment by eliminating manual configuration.

1. Finance Application

A lightweight Finance web application is developed using Python Flask. It provides basic endpoints such as balance and transaction details and serves as the main application deployed in the system.

2. Containerization using Docker

The application is packaged into a Docker container to maintain a consistent runtime environment. Docker installs dependencies, exposes port 5000, and allows the application to run identically on local and cloud systems.

3. Infrastructure Provisioning using Terraform

Terraform automates AWS infrastructure creation using code. It provisions:

- VPC
- Public Subnet
- Internet Gateway
- Route Table
- Security Group
- EC2 Instance

This enables repeatable and scalable infrastructure deployment.

4. Automatic Deployment via EC2 User-Data

During EC2 creation, a user-data script automatically installs Docker, pulls the application image from Docker Hub, and runs the container. This removes the need for manual server setup.

5. CI/CD Automation with GitHub Actions

GitHub Actions automates the deployment pipeline. On every code push:

- Docker image is built
- Image is pushed to Docker Hub
- Terraform applies infrastructure updates

This ensures continuous integration and deployment.

6. Workflow (End-to-End Process)

Developer → Git Push → GitHub Actions → Docker Build & Push → Terraform Apply → AWS Infrastructure Creation → EC2 Auto Setup → Application Deployment

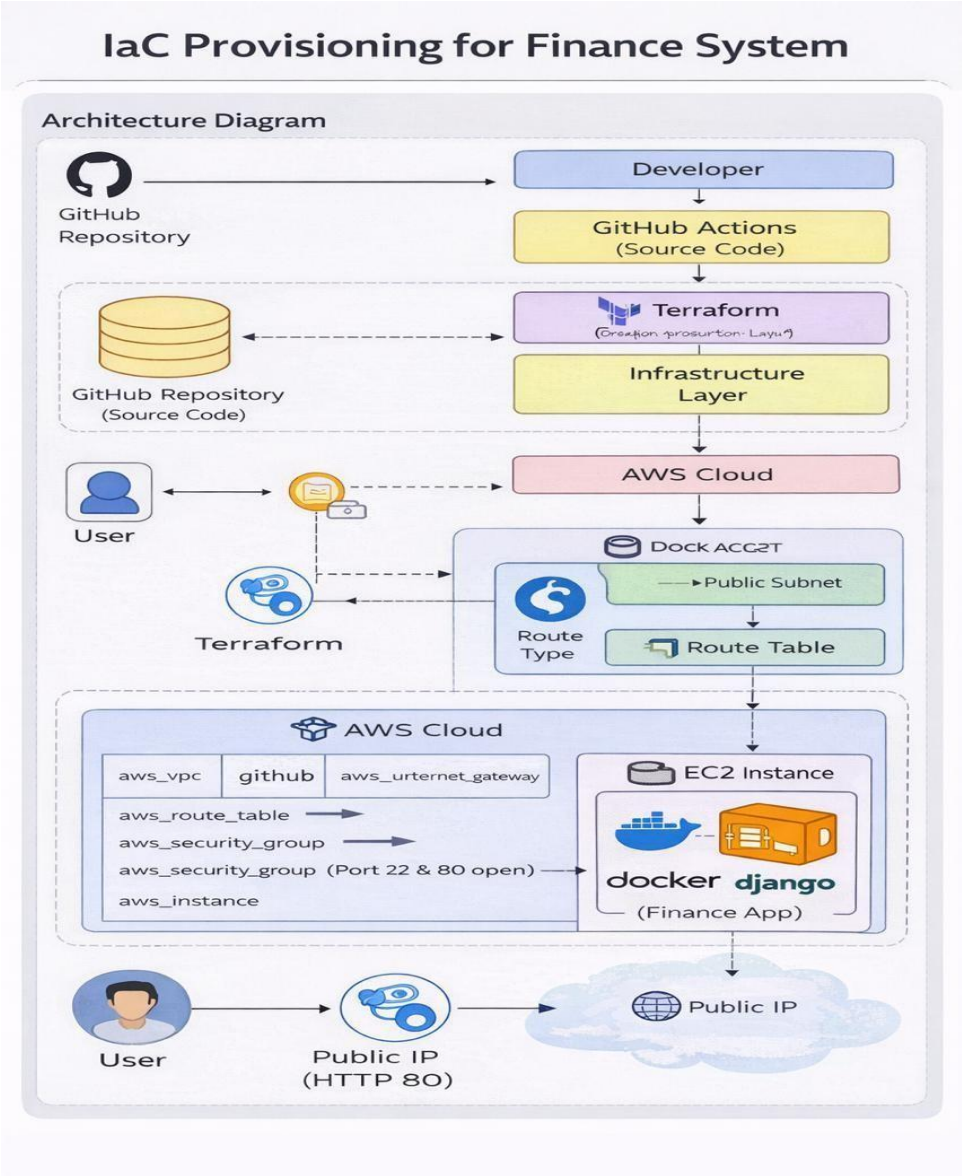

Fig. 1

Results & Output

1. Accesible through Public IP

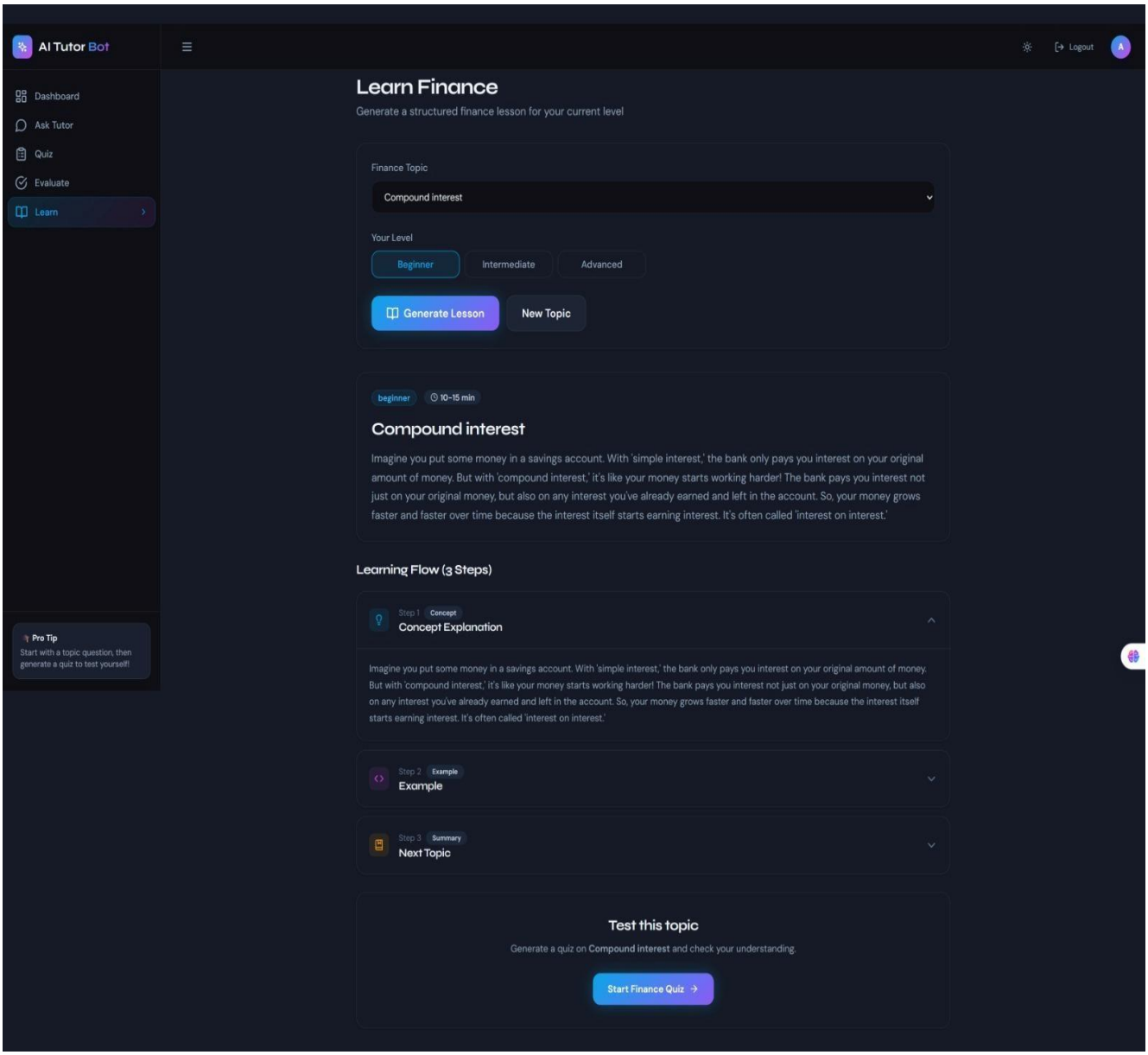

Fig. 1.1

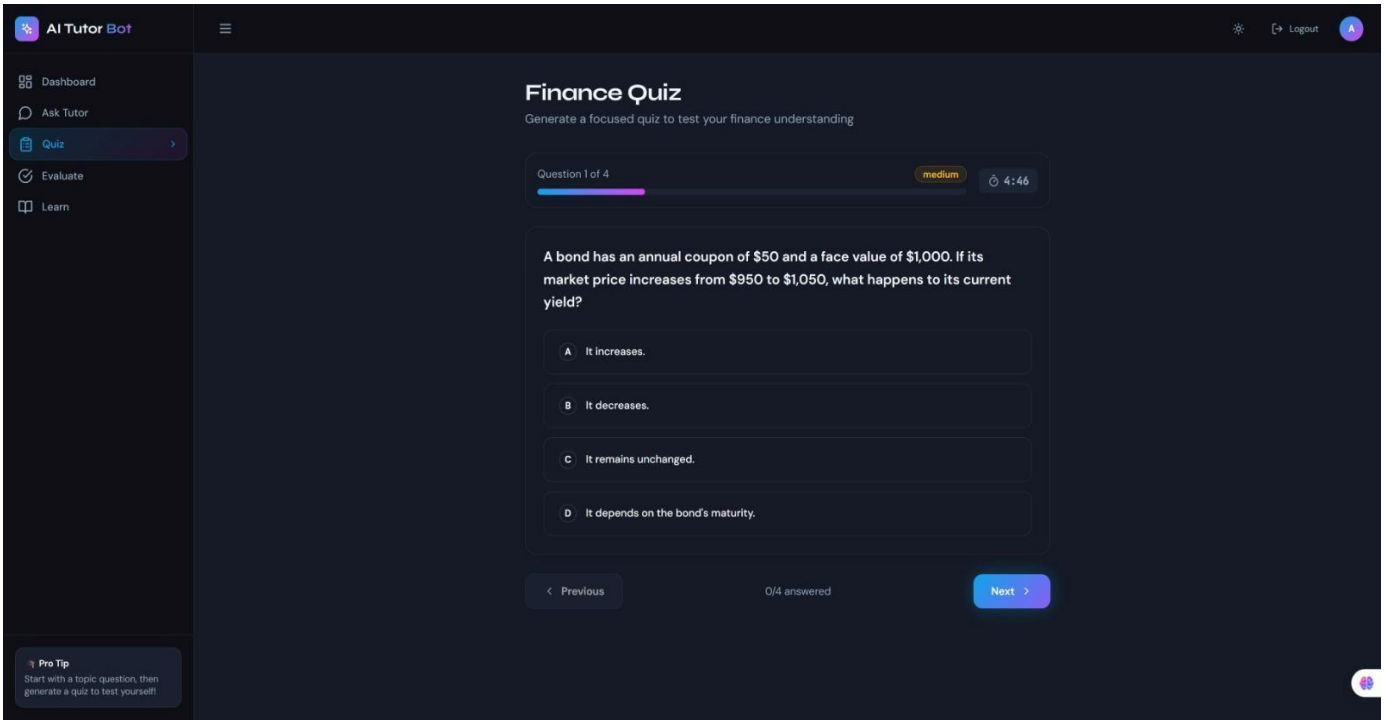
AI Tutor Bot
Logout
Dashboard
Ask Tutor
Quiz
Evaluate
Learn
Finance Quiz
Generate a focused quiz to test your finance understanding
Question 1 of 4
medium
4:46
A bond has an annual coupon of $50 and a face value of $1,000. If its market price increases from $950 to $1,050, what happens to its current yield?
A It increases.
B It decreases.
C It remains unchanged.
D It depends on the bond's maturity.
Previous
0/4 answered
Next
Pro Tip
Start with a topic question, then generate a quiz to test yourself!

Fig. 1.2

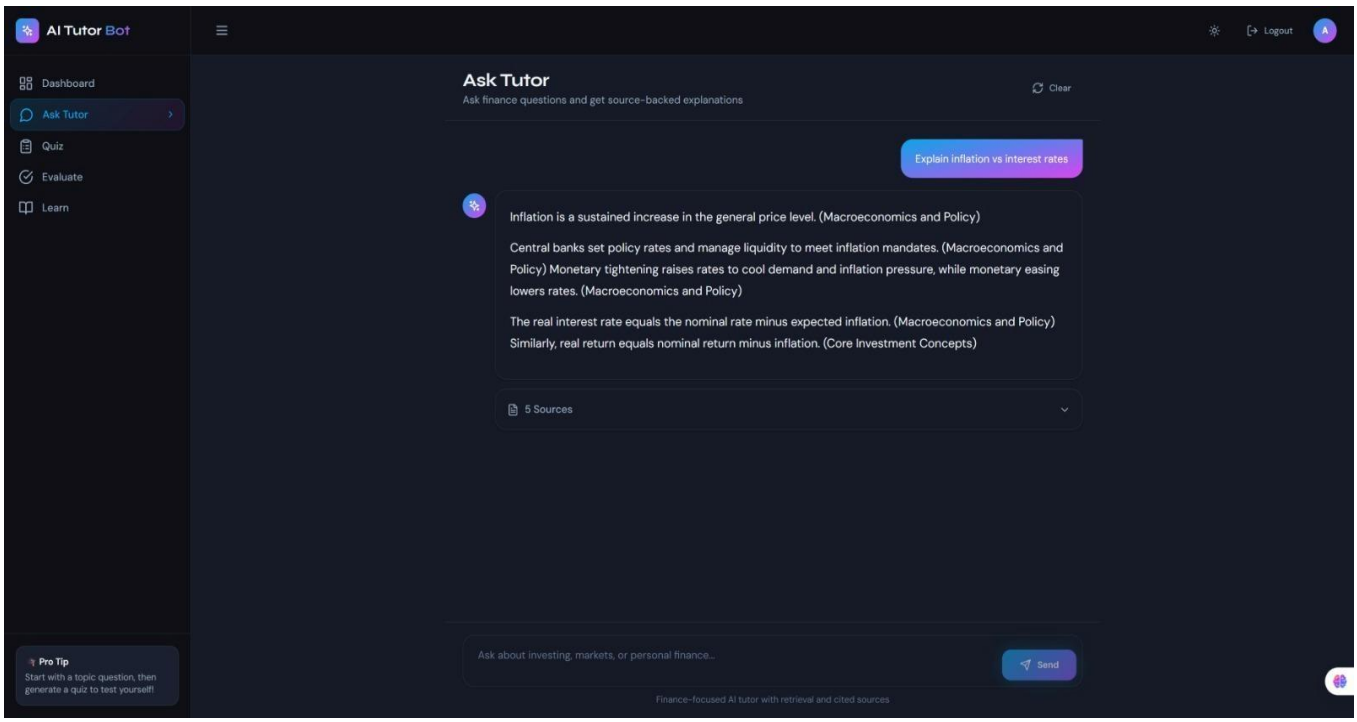
AI Tutor Bot
Logout
Dashboard
Ask Tutor
Quiz
Evaluate
Learn
Ask Tutor
Ask finance questions and get source-backed explanations
Clear
Explain inflation vs interest rates
Inflation is a sustained increase in the general price level. (Macroeconomics and Policy)
Central banks set policy rates and manage liquidity to meet inflation mandates. (Macroeconomics and Policy) Monetary tightening raises rates to cool demand and inflation pressure, while monetary easing lowers rates. (Macroeconomics and Policy)
The real interest rate equals the nominal rate minus expected inflation. (Macroeconomics and Policy) Similarly, real return equals nominal return minus inflation. (Core Investment Concepts)
5 Sources
Ask about investing, markets, or personal finance...
Send
Finance-focused AI tutor with retrieval and cited sources
Pro Tip
Start with a topic question, then generate a quiz to test yourself!

Fig. 1.3

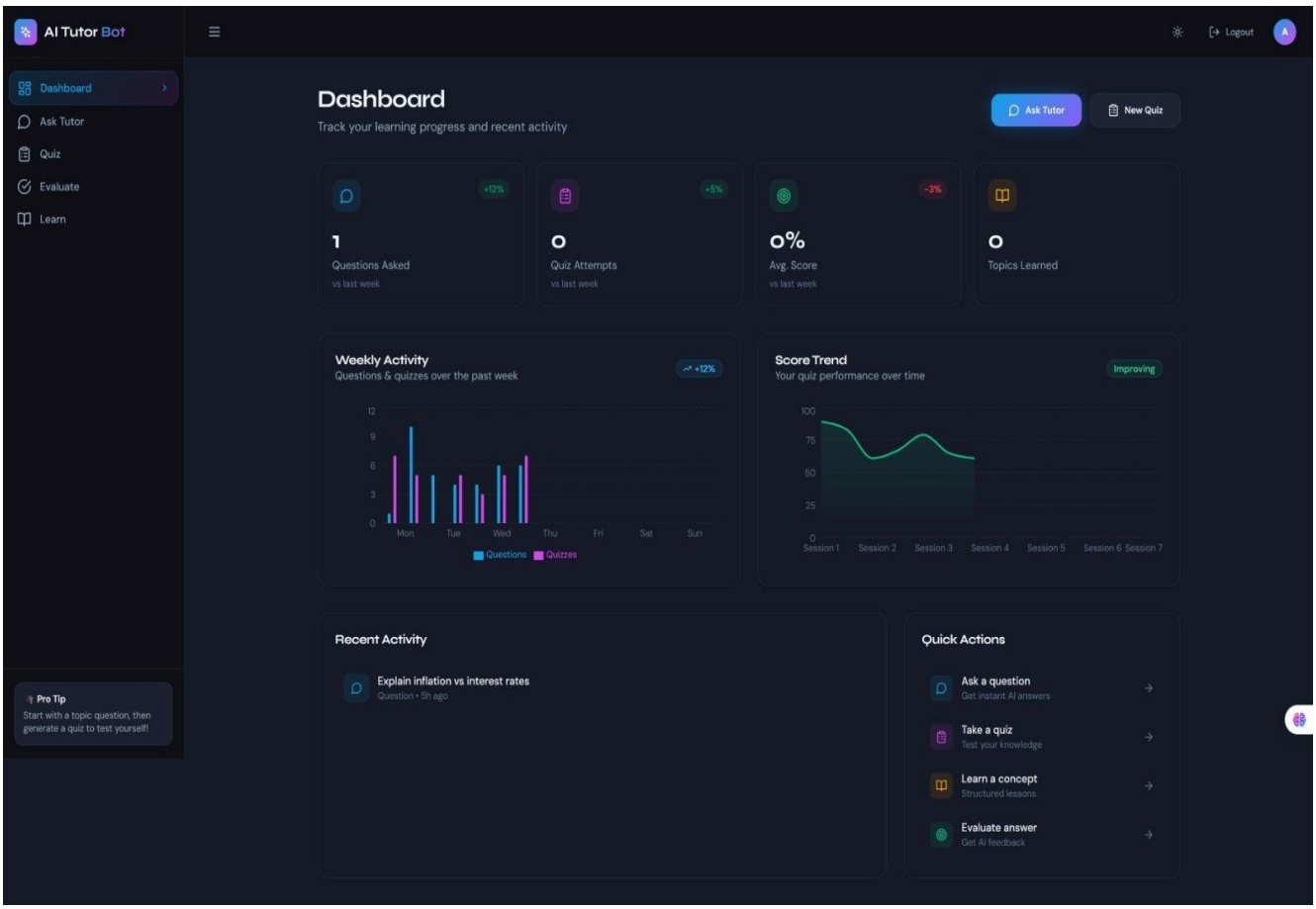
AI Tutor Bot
Logout
Dashboard
Ask Tutor
Quiz
Evaluate
Learn
Dashboard
Track your learning progress and recent activity
Ask Tutor
New Quiz
+12%
+5%
-3%
1
Questions Asked
vs last week
0
Quiz Attempts
vs last week
0%
Avg. Score
vs last week
0
Topics Learned
Weekly Activity
Questions & quizzes over the past week
+12%
12
9
6
3
0
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Questions
Quizzes
Score Trend
Your quiz performance over time
Improving
100
75
50
25
0
Session 1
Session 2
Session 3
Session 4
Session 5
Session 6
Session 7
Pro Tip
Start with a topic question, then generate a quiz to test yourself!
Recent Activity
Explain inflation vs interest rates
Question • 5h ago
Quick Actions
Ask a question
Get instant AI answers
Take a quiz
Test your knowledge
Learn a concept
Structured lessons
Evaluate answer
Get AI feedback

Fig. 1.4

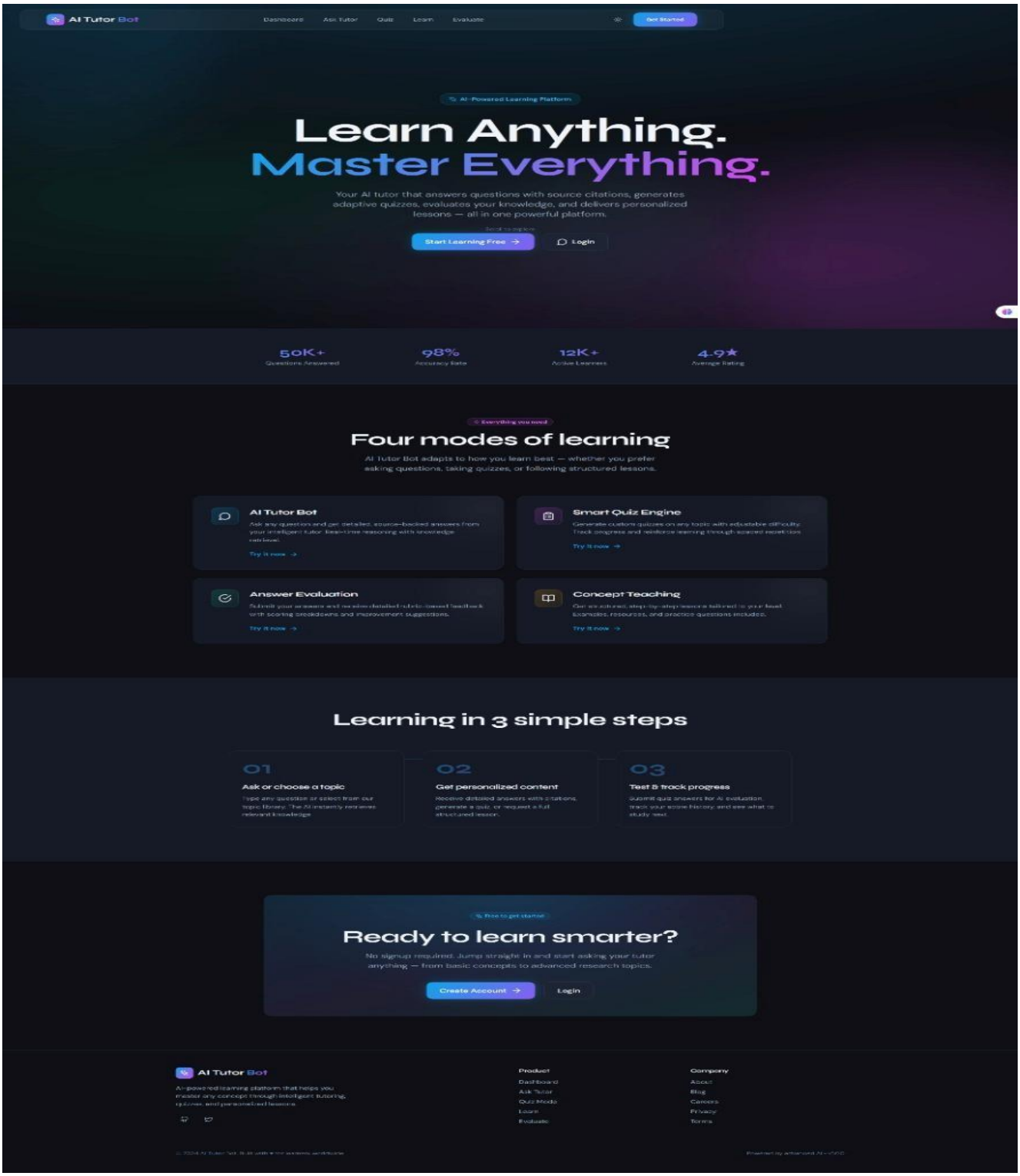
AI Tutor Bot
Dashboard
Ask Tutor
Quiz
Learn
Evaluate
Get Started
Learn Anything.
Master Everything.
Start Learning Free
Login
50K+
98%
12K+
4.9★
Four modes of learning
AI Tutor Bot
Smart Quiz Engine
Answer Evaluation
Concept Teaching
Learning in 3 simple steps
01
Ask or choose a topic
02
Get personalized content
03
Test & track progress
Ready to learn smarter?
Create Account
Login
AI Tutor Bot
Product
Company

Fig. 1.5

2. Containerization using Docker

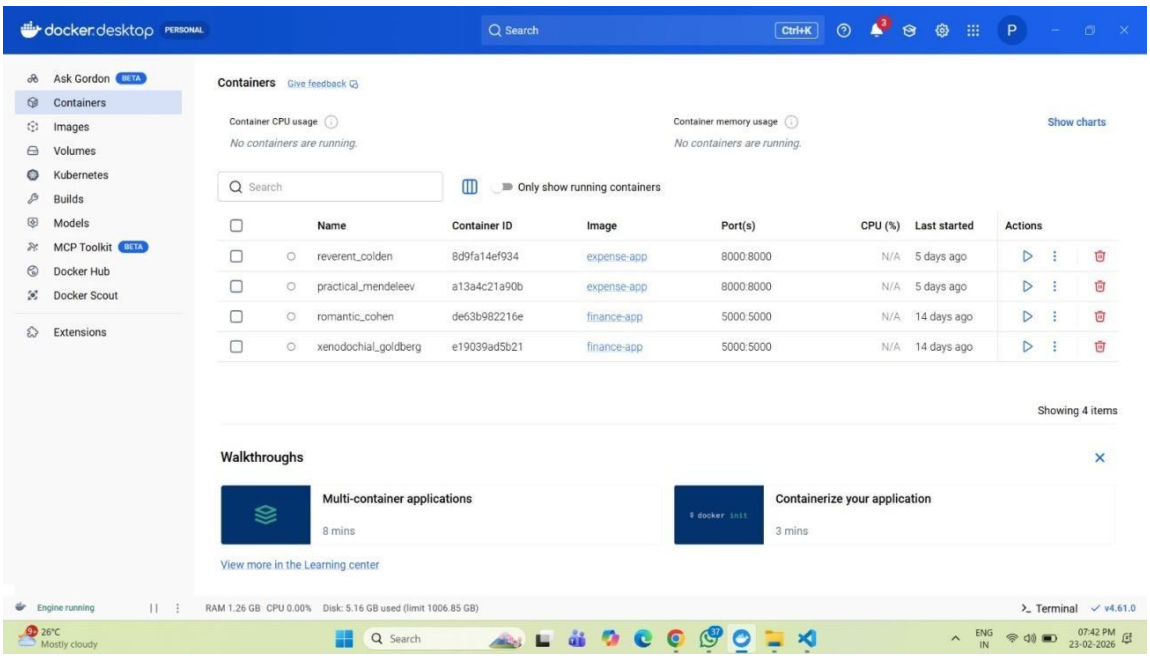

Fig. 2.1

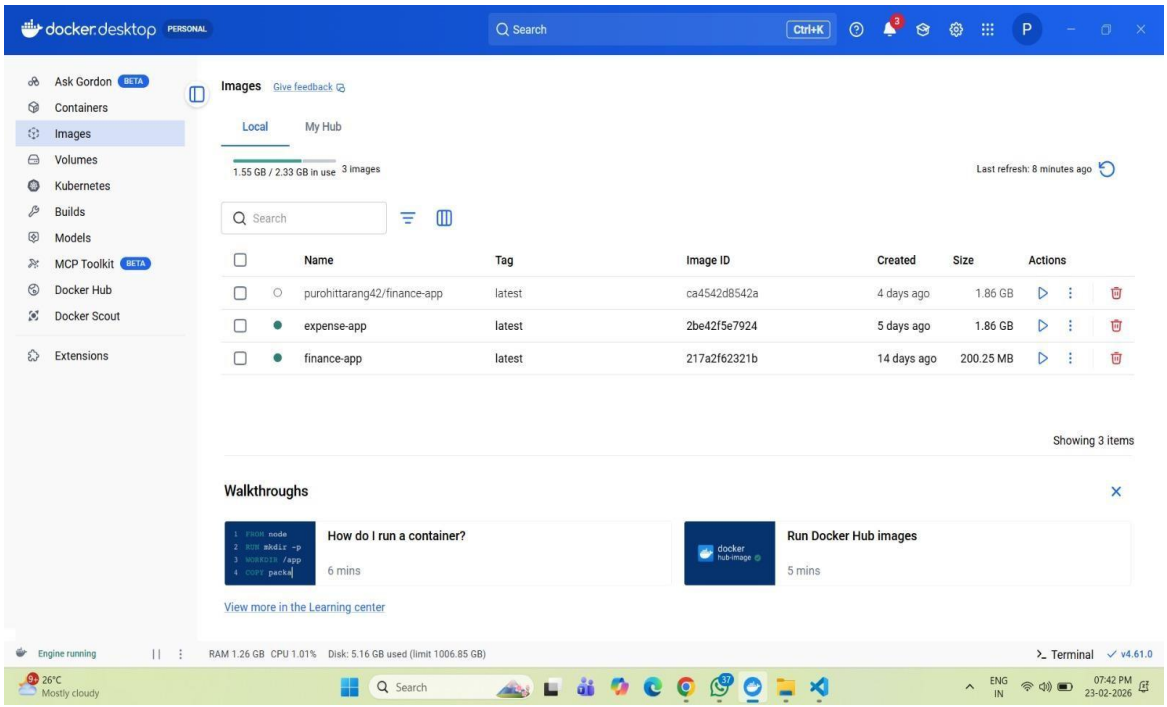

Fig. 2.2

3. Resource Provisioning through Terraform

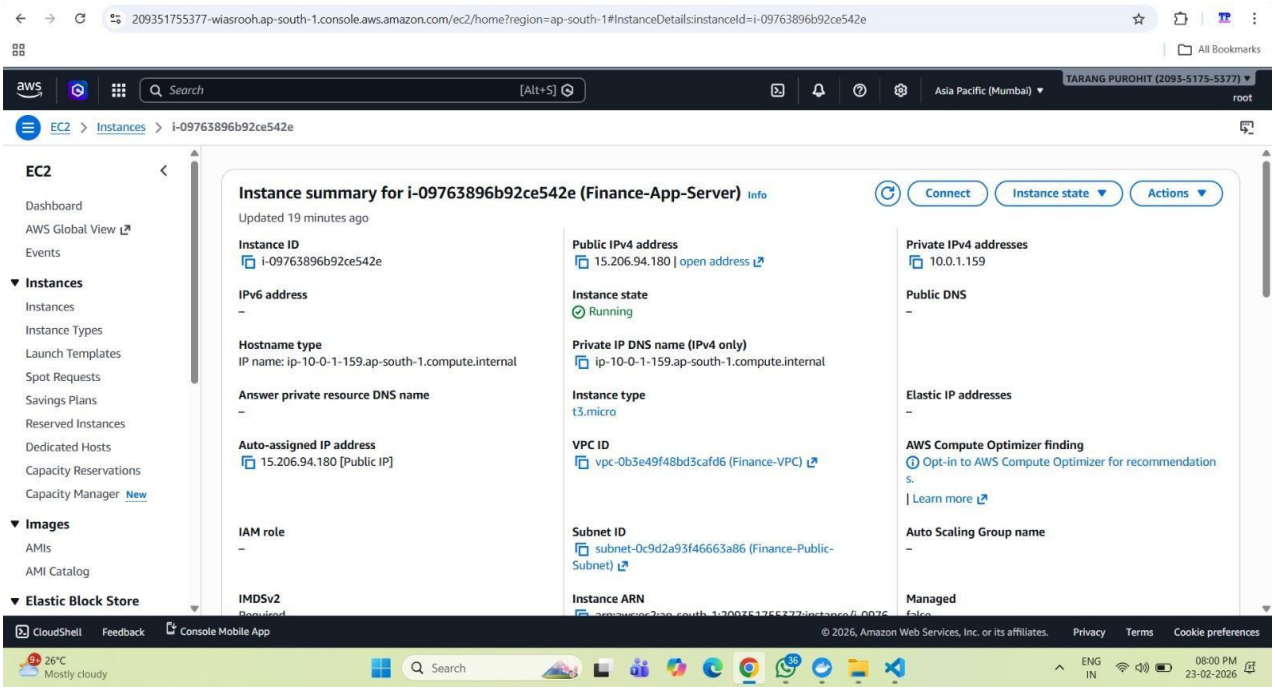

Fig. 3.1

Conclusion

This project successfully demonstrates the automation of infrastructure provisioning and application deployment for a Finance system using Infrastructure as Code (IaC). By integrating Docker, Terraform, AWS, and CI/CD automation, the project eliminates manual configuration processes and ensures consistent, repeatable, and reliable deployments. The automated workflow enables faster environment setup, reduces human errors, and improves deployment efficiency. Overall, the solution showcases modern DevOps practices for building scalable and maintainable cloud-based applications.

Future Scope & Enhancements

- **Implementation of Application Load Balancer (ALB):**
  An ALB can be added to distribute incoming traffic across multiple application instances, improving performance, availability, and fault tolerance.
- **Integration of Auto Scaling Groups:**
  Auto Scaling can automatically increase or decrease EC2 instances based on workload demand, ensuring efficient resource utilization and system scalability.
- **Migration to Amazon RDS:**
  The application database can be moved to Amazon RDS to provide managed, secure, and persistent database storage with automated backups and maintenance.
- **Use of S3 Remote Backend for Terraform:**
  Terraform state files can be stored in Amazon S3 to enable secure state management, version control, and collaboration among team members.
- **Implementation of IAM Role-Based Access Control:**
  IAM roles and policies can be used to restrict permissions and enhance security by granting least-privilege access to AWS resources.
- **Deployment Across Multi-AZ Architecture:**
  Deploying resources across multiple Availability Zones will improve system reliability and ensure high availability during failures.
- **Monitoring and Logging using AWS CloudWatch:**
  AWS CloudWatch can be integrated for real-time monitoring, logging, and alerting to track application performance and detect issues proactively.

.